

You said

Selected files all

The Master Prompt to Copy/Paste into Workik AI Whenever you upload a new controller or file to Workik, paste this exact prompt into the chat sidebar:

Prompt: "Act as a Senior .NET Core Developer and Debugging Expert. I have attached a file/controller from my project. I need a deep-dive, line-by-line, and word-by-word explanation. For every single line, please provide:

Line Purpose: What exactly does this line of code do?

Keyword Breakdown: Break down specific technical keywords used in that line (e.g., async, [FromBody], Task<IActionResult>).

Cross-File References (Debugging Track): Tell me which other projects, folders, or files in a standard Clean Architecture setup this line links to (e.g., Where is the DTO folder? Where is the Interface file? Where is the Service implementation?).

Data Flow: What happens to the data at this exact step?"

FOR EXAMPLE >>>

2. HRMS.API/Controllers/AuthController.cs

Handles user registration, login, and token management.

Line: [ApiController] [Route("api/[controller]")]

Line Purpose: Decorates the class as a Web API controller with a route based on the class name (api/auth).

Keyword Breakdown: [ApiController] (enables automatic model validation and HTTP 400 responses), [Route] (defines the URL path).

Cross-File References: Links to Program.cs routing configuration.

Data Flow: Directs incoming URL requests to this class.

Line: public async Task<IActionResult> Login([FromBody] LoginDto dto)

Line Purpose: Endpoint for user authentication.

Keyword Breakdown: IActionResult (flexible return type for HTTP statuses like Ok, BadRequest), [FromBody] (forces data to be read from the JSON body).

Cross-File References: LoginDto is in HRMS.Application/DTOs/Auth/.

Data Flow: JSON from the client is mapped into the dto object.

Line: var result = await _authService.LoginAsync(dto);

Line Purpose: Calls the business logic layer to verify credentials.

Keyword Breakdown: _authService (the injected interface implementation), await (waits for the service to finish).

Cross-File References: Interface: IAuthService.cs, Implementation: AuthService.cs in HRMS.Application.

Data Flow: Sends user credentials to the service and receives an ApiResponse containing a JWT.

so in my given files you give me according to my given above prompt and like example so we moved one by one files sequencial

>>>IT'S MY INFRESTRUCTURE FOLDER'S FILE

Gemini said

Here is the comprehensive, line-by-line, and word-by-word breakdown of your **Infrastructure Layer** files, structured exactly like your example.

We will proceed through the files sequentially, starting with your database context.

File 1: `ApplicationDbContext.cs`

`using HRMS.Domain.Entities;`

- **Line Purpose:** Imports the namespace containing domain models (like `Organization`, `Employee`, `ApplicationUser`) so they can be referenced to configure database tables.
- **Keyword Breakdown:** `using` (directive used to allow types in a namespace to be used without full qualification).
- **Cross-File References:** Links to the **Domain** project, specifically files inside the `Entities/` folder (e.g., `Employee.cs`, `Organization.cs`, `ApplicationUser.cs`).
- **Data Flow:** None (compile-time namespace inclusion).

`using Microsoft.AspNetCore.Identity;`

- **Line Purpose:** Imports ASP.NET Core Identity types to enable management of system roles, tokens, and claims.
- **Keyword Breakdown:** `Microsoft.AspNetCore.Identity` (the framework's identity management library).
- **Cross-File References:** Brought in via NuGet package. Used directly here to declare `IdentityRole<Guid>`, `IdentityUserRole<Guid>`, etc.
- **Data Flow:** None.

`using Microsoft.AspNetCore.Identity.EntityFrameworkCore;`

- **Line Purpose:** Imports the Entity Framework Core extension for Identity, providing the base `IdentityDbContext` class.

- **Keyword Breakdown:** `EntityFrameworkCore` (an object-relational mapper for .NET).
- **Cross-File References:** Brought in via NuGet package. Links this file structurally to the EF Core ecosystem.
- **Data Flow:** None.

```
using Microsoft.EntityFrameworkCore;
```

- **Line Purpose:** Imports the core Entity Framework library required for database modeling, configuration APIs, and querying tracking.
- **Keyword Breakdown:** `DbContext` (the primary class responsible for interacting with the database).
- **Cross-File References:** Essential base dependency for configuring database entities throughout this layer.
- **Data Flow:** None.

```
namespace HRMS.Infrastructure.Data;
```

- **Line Purpose:** Declares the logical namespace organizational boundary for database context files.
- **Keyword Breakdown:** `namespace` (keyword declaring a scope that contains a set of related objects), file-scoped namespace syntax (ends with a semicolon).
- **Cross-File References:** Referenced by `DependencyInjection.cs` to register this context into the application's service container.
- **Data Flow:** None.

```
public class ApplicationDbContext : IdentityDbContext<ApplicationUser, IdentityRole<Guid>, Guid>
```

- **Line Purpose:** Defines your database context class, inheriting from `IdentityDbContext` configured to use `Guid` as the primary key type for all built-in identity tables instead of the default strings.
- **Keyword Breakdown:** `public` (accessible outside this assembly), `class` (defines a reference type), `:` (inherits from), `IdentityDbContext<...>` (a built-in EF context that includes tables for users, roles, claims, and tokens), `Guid` (Globally Unique Identifier structural type).
- **Cross-File References:** Inherits custom user definition from `ApplicationUser` in `HRMS.Domain.Entities`.
- **Data Flow:** Acts as the primary transactional bridge between your C# backend application entities and the underlying PostgreSQL database instances.

```
public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options)
```

- **Line Purpose:** Constructor that accepts runtime configurations (like connection strings and database providers) and passes them up to the underlying EF base framework.
- **Keyword Breakdown:** `DbContextOptions<T>` (carries configuration information for a DbContext), `base(options)` (invokes the parent class's constructor with the provided argument).
- **Cross-File References:** Invoked implicitly by the Web API's dependency injection engine configured in `DependencyInjection.cs`.
- **Data Flow:** Receives database connection configurations at application startup and instantiates the context tracking pipeline.

```
public DbSet<Organization> Organizations { get; set; } = null!;
```

- **Line Purpose:** Declares a queryable database collection matching the `Organization` table structure.
- **Keyword Breakdown:** `DbSet<T>` (represents a collection of a given entity type in the context for CRUD operations), `= null!` (null-forgiving operator telling the compiler that EF will initialize this property at runtime).
- **Cross-File References:** Maps directly to `Organization.cs` inside the **Domain** entities project.

- **Data Flow:** Serves as the main gateway for reading/writing organization rows via EF Core (though your repositories bypass this with Dapper, this handles DB schema generations).

```
public DbSet<Employee> Employees { get; set; } = null!;
```

- **Line Purpose:** Declares a queryable database collection matching the `Employee` entity table schema.
- **Keyword Breakdown:** `DbSet<Employee>` (the collection representation of employees).
- **Cross-File References:** Maps directly to `Employee.cs` in the **Domain** entities layer.
- **Data Flow:** Facilitates schema generation and cascade constraint mapping for employees.

```
public DbSet<RefreshToken> RefreshTokens { get; set; } = null!;
```

- **Line Purpose:** Declares a database collection mapping for application user security refresh tokens.
- **Keyword Breakdown:** `DbSet<RefreshToken>` (collection representation of persistent authentication tokens).
- **Cross-File References:** Links directly to `RefreshToken.cs` in the **Domain** entities layer.
- **Data Flow:** Used to establish structural schema setups for checking token lifecycles.

```
protected override void OnModelCreating(ModelBuilder builder)
```

- **Line Purpose:** Overrides the base configuration method to apply custom database mapping rules like customized table names, explicit field lengths, and foreign key rules.
- **Keyword Breakdown:** `protected` (accessible within this class and derived classes), `override` (extends or modifies the abstract or virtual implementation of an inherited method), `ModelBuilder` (the API fluent builder used to configure database mappings).
- **Cross-File References:** Runs internally by EF Core during database initial migrations or runtime model builds.
- **Data Flow:** Constructs the operational metadata map of how memory objects transform into SQL database entities.

```
base.OnModelCreating(builder);
```

- **Line Purpose:** Runs the base Identity configuration mapping logic, ensuring that default identity schemas are initialized before applying overrides.
- **Keyword Breakdown:** `base` (accesses members of the base class from within a derived class).
- **Cross-File References:** Executes code defined deep inside the `Microsoft.AspNetCore.Identity.EntityFrameworkCore` package.
- **Data Flow:** Configures default structural tables for Identity (AspNetUsers, AspNetRoles, etc.) directly into the builder pipeline.

```
builder.Entity<ApplicationUser>(entity => { entity.ToTable("Users"); ... });
```

- **Line Purpose:** Customizes the Identity user table metadata by changing its default name from `AspNetUsers` to `Users` and mapping its custom organization relationships.
- **Keyword Breakdown:** `Entity<T>` (returns an object that can be used to configure a given entity type), `ToTable("String")` (specifies the database table name).
- **Cross-File References:** Maps configuration onto `ApplicationUser.cs` in **Domain**.
- **Data Flow:** Directs EF Core migrations to output SQL table structures explicitly named `Users`.

```
entity.HasOne(u => u.Organization).WithMany(o => o.Users).HasForeignKey(u =>
u.OrganizationId).OnDelete(DeleteBehavior.Restrict);
```

- **Line Purpose:** Defines a **One-to-Many** database relationship where an Organization can contain many Users, enforcing a Restrict deletion rule.
- **Keyword Breakdown:** `HasOne` (configures a navigation property to a single entity), `WithMany` (configures the inverse collections property), `HasForeignKey` (identifies the property acting as the relational key), `DeleteBehavior.Restrict` (prevents deletion of an Organization if dependent User entities exist).
- **Cross-File References:** Maps links between `ApplicationUser.cs` and `Organization.cs` inside **Domain**.
- **Data Flow:** Enforces referential integrity directly at the database engine level during deletions.

```
builder.Entity<IdentityRole<Guid>>(entity => { entity.ToTable("Roles"); });
```

- **Line Purpose:** Standardizes database names by renaming the default identity table `AspNetRoles` to `Roles`.
- **Keyword Breakdown:** `IdentityRole<Guid>` (the framework's built-in system role structure using unique GUID keys).
- **Cross-File References:** Linked during role seeding execution inside `IdentitySeed.cs`.
- **Data Flow:** Instructs the database provider to construct or lookup system security roles using the `Roles` table name.

```
builder.Entity<IdentityUserRole<Guid>>(entity => { entity.ToTable("UserRoles"); });
```

- **Line Purpose:** Customizes the join table name connecting Users and Roles together, renaming it from `AspNetUserRoles` to `UserRoles`.
- **Keyword Breakdown:** `IdentityUserRole<Guid>` (the built-in mapping entity for user-to-role relationships).
- **Cross-File References:** Implicitly updated whenever roles are added or verified via a custom role/user manager setup.
- **Data Flow:** Dictates the relational bridge lookups when querying what roles a specific user possesses.

```
builder.Entity<IdentityUserClaim<Guid>>(entity => { entity.ToTable("UserClaims"); });
```

- **Line Purpose:** Renames the default identity claims table from `AspNetUserClaims` to `UserClaims`.
- **Keyword Breakdown:** `IdentityUserClaim<Guid>` (resembles specific security claims issued to individual users).
- **Cross-File References:** Used during auth security token construction in services.
- **Data Flow:** Maps custom metadata statements asserted about a user into a dedicated table.

```
builder.Entity<IdentityUserLogin<Guid>>(entity => { entity.ToTable("UserLogins"); });
```

- **Line Purpose:** Renames the default identity external login audit table from `AspNetUserLogins` to `UserLogins`.
- **Keyword Breakdown:** `IdentityUserLogin<Guid>` (holds third-party provider login metadata links).
- **Cross-File References:** Part of the extended core identity suite architecture.
- **Data Flow:** Stores records mapping external identity providers to local user identifiers.

```
builder.Entity<IdentityRoleClaim<Guid>>(entity => { entity.ToTable("RoleClaims"); });
```

- **Line Purpose:** Renames the role permission metadata table from `AspNetRoleClaims` to `RoleClaims`.
- **Keyword Breakdown:** `IdentityRoleClaim<Guid>` (represents permissions attached globally to corporate system roles).
- **Cross-File References:** Utilized during global role authorization checks.
- **Data Flow:** Holds configuration claims inherited by any user assigned to the associated role.

```
builder.Entity<IdentityUserToken<Guid>>(entity => { entity.ToTable("UserTokens"); });
```

- **Line Purpose:** Renames the low-level utility tokens table from `AspNetUserTokens` to `UserTokens`.
- **Keyword Breakdown:** `IdentityUserToken<Guid>` (stores application internal utility security strings like password resets).
- **Cross-File References:** Interacts with default token generation engines.
- **Data Flow:** Manages internal system automation and verification lifecycle strings.

```
builder.Entity<Organization>(entity => { ... });
```

- **Line Purpose:** Groups the schema configurations applied specifically to the structural `Organization` entity.
- **Cross-File References:** Targets database representations of `Organization.cs` from your **Domain** layer.
- **Data Flow:** Pipelines explicit structural limits into the database building engine.

```
entity.ToTable("Organizations"); entity.HasKey(o => o.Id);
```

- **Line Purpose:** Explicitly binds the model to a database table named `Organizations` and defines the `Id` column as its primary key.
- **Keyword Breakdown:** `HasKey` (configures the property or properties that serve as the unique key for this entity type).
- **Cross-File References:** Matches property variables in `Organization.cs`.
- **Data Flow:** Assures efficient unique index lookups are physically created on the database level.

```
entity.Property(o => o.Name).IsRequired().HasMaxLength(250);
```

- **Line Purpose:** Places constraints on the organization name field: it must not be null and cannot exceed 250 characters.
- **Keyword Breakdown:** `Property` (targets attributes for configuration), `IsRequired` (makes field NOT NULL in SQL), `HasMaxLength` (sets maximum length constraints).
- **Cross-File References:** Enforces rules for corporate configurations across the business layer.
- **Data Flow:** Raises database-level validation errors if a client tries to save an empty or overly long organization name string.

```
entity.Property(o => o.Address).HasMaxLength(500);
```

- **Line Purpose:** Configures the optional physical address field constraint up to a limit of 500 characters.
- **Cross-File References:** Corresponds to data structures on `Organization.cs`.
- **Data Flow:** Limits the maximum storage allocation block size for address text records.

```
builder.Entity<Employee>(entity => { ... });
```

- **Line Purpose:** Groups schema-level metadata configuration rules specifically targeting your system `Employee` persistence records.
- **Cross-File References:** Formulates blueprints utilizing `Employee.cs` in the **Domain** layer.

```
entity.ToTable("Employees"); entity.HasKey(e => e.Id);
```

- **Line Purpose:** Explicitly targets the table string identifier name as `Employees` and designates `Id` as its primary indexing entity key.
- **Cross-File References:** Aligns database attributes with `Employee.cs` specifications.

```
entity.Property(e => e.FirstName).IsRequired().HasMaxLength(100);
```

- **Line Purpose:** Sets constraints for the system employee first name: non-nullable, maximum 100 character array block.
- **Cross-File References:** Maps logic to properties declared in `Employee.cs`.

```
entity.Property(e => e.LastName).IsRequired().HasMaxLength(100);
```

- **Line Purpose:** Sets constraints for the system employee last name: non-nullable, maximum 100 character array block.
- **Cross-File References:** Maps logic to properties declared in `Employee.cs`.

```
entity.Property(e => e.Email).IsRequired().HasMaxLength(150);
```

- **Line Purpose:** Sets data rules for the system employee email tracking field: non-nullable, maximum 150 characters.
- **Cross-File References:** Maps logic to properties declared in `Employee.cs`.

```
entity.Property(e => e.PhoneNumber).HasMaxLength(20);
```

- **Line Purpose:** Sets field length constraints on employee corporate phone text data up to a maximum of 20 characters.
- **Cross-File References:** Maps logic to properties declared in `Employee.cs`.

```
entity.Property(e => e.Address).HasMaxLength(500);
```

- **Line Purpose:** Maps maximum data configuration bounds allowing up to 500 characters for residential records storage.
- **Cross-File References:** Maps logic to properties declared in `Employee.cs`.

```
entity.HasOne(e => e.Organization).WithMany(o => o.Employees).HasForeignKey(e =>
e.OrganizationId).OnDelete(DeleteBehavior.Cascade);
```

- **Line Purpose:** Builds a relationship where an organization possesses multiple employees. If an organization is deleted, all associated employee rows are cascade deleted.
- **Keyword Breakdown:** `DeleteBehavior.Cascade` (dependent entities are deleted automatically when the principal entity is deleted).
- **Cross-File References:** Connects `Employee.cs` data records to tracking collections inside `Organization.cs`.
- **Data Flow:** Triggers immediate cleanup across dependent database child tables if an operational parent entity row gets purged.

```
entity.HasOne(e => e.CreatedByUser).WithMany().HasForeignKey(e =>
e.CreatedBy).OnDelete(DeleteBehavior.Restrict);
```

- **Line Purpose:** Establishes an audit relationship tracking which user created an employee record. It prevents deletion of that user if they are linked to employee records.
- **Cross-File References:** Ties employee audit pointers (`CreatedBy` property) directly to individual `ApplicationUser` tables.
- **Data Flow:** Protects system historical data integrity by preventing the accidental deletion of administrative users who created employee profiles.

```
builder.Entity<RefreshToken>(entity => { ... });
```

- **Line Purpose:** Groups the schema configurations applied specifically to the security `RefreshToken` database model.
- **Cross-File References:** Shapes persistence definitions leveraging `RefreshToken.cs` definitions from **Domain**.

```
entity.ToTable("RefreshTokens"); entity.HasKey(rt => rt.Id);
```

- **Line Purpose:** Configures the database destination name to `RefreshTokens` and designates its unique lookup key attribute as `Id`.
- **Cross-File References:** Aligns mappings to field parameters defined inside `RefreshToken.cs`.

```
entity.Property(rt => rt.Token).IsRequired().HasMaxLength(500);
```

- **Line Purpose:** Enforces security system constraints requiring token hashes to be non-nullable strings capped at 500 characters max.
- **Cross-File References:** Restricts structures matching parameters of `RefreshToken.cs`.

```
entity.HasOne(rt => rt.User).WithMany(u => u.RefreshTokens).HasForeignKey(rt =>
rt.UserId).OnDelete(DeleteBehavior.Cascade);
```

- **Line Purpose:** Configures a **One-to-Many** mapping where users own multiple active session tokens. If a user record is removed, all their active session refresh tokens are cascade deleted.
- **Cross-File References:** Establishes cross-table validation logic bridging `RefreshToken.cs` and `ApplicationUser.cs`.
- **Data Flow:** Automatically purges operational login tokens from the database when a user account is deleted.

File 2: `IdentitySeed.cs`

```
using HRMS.Domain.Enums;
```

- **Line Purpose:** Imports the enum definitions namespace to access system role constants.
- **Keyword Breakdown:** `using` (directive referencing access to scopes).
- **Cross-File References:** Links directly to `UserRole.cs` inside the **Domain** project layer's `Enums/` folder.
- **Data Flow:** None (compile-time constant access).

```
using Microsoft.AspNetCore.Identity;
```

- **Line Purpose:** Directs the compiler to pull in identity framework components needed to manage and validate user roles.
- **Keyword Breakdown:** `Identity` (ASP.NET user management architecture tool sets).
- **Cross-File References:** Framework engine components pulled from package assemblies.
- **Data Flow:** None.

```
using Microsoft.Extensions.DependencyInjection;
```

- **Line Purpose:** Imports classes that resolve objects from the application's central Dependency Injection (DI) container.
- **Cross-File References:** Connects this script directly to system container resolution operations.
- **Data Flow:** None.

```
namespace HRMS.Infrastructure.Identity;
```

- **Line Purpose:** Establishes the code file logical assembly organization name scope.
- **Cross-File References:** Scoped area containing application data seeding mechanisms.

```
public static class IdentitySeed
```

- **Line Purpose:** Declares a global helper class containing setup methods that run once during application initialization.
- **Keyword Breakdown:** `static` (declares a class that cannot be instantiated and can only contain static members).
- **Cross-File References:** Invoked at the application root during cold-start configurations (typically inside `Program.cs`).
- **Data Flow:** Instantiates the foundational security roles required for user authorization.

```
public static async Task SeedRolesAsync(IServiceProvider serviceProvider)
```

- **Line Purpose:** Asynchronously initializes the system database with standard access roles if they do not already exist.
- **Keyword Breakdown:** `async` (marks method as asynchronous allowing use of await), `Task` (represents an asynchronous operation that returns no value), `IServiceProvider` (defines a mechanism for retrieving a service object).
- **Cross-File References:** Called from `Program.cs`. Receives the running application's service provider context.
- **Data Flow:** Connects to the active application context scope to inspect and provision system security records.

```
var roleManager = serviceProvider.GetRequiredService<RoleManager<IdentityRole<Guid>>>();
```

- **Line Purpose:** Resolves the framework's Role Manager instance from the Dependency Injection container to manage role records.
- **Keyword Breakdown:** `GetRequiredService<T>` (retrieves a service of type T from the provider, throwing an exception if it doesn't exist), `RoleManager<T>` (provides APIs for managing roles in a persistence store).
- **Cross-File References:** Leverages configuration details instantiated in `DependencyInjection.cs`.
- **Data Flow:** Pulls the active connection context role engine from the DI container to execute database queries.

```
string[] roles = { UserRole.Admin, UserRole.HR };
```

- **Line Purpose:** Creates an array of the required system roles using constants from the Domain layer.
- **Keyword Breakdown:** `string[]` (defines an array of string items), `UserRole.Admin` (constant string tracking values).
- **Cross-File References:** Pulls role values from `UserRole.cs` inside the **Domain** Enums layout.
- **Data Flow:** Loads administrative role name configurations into local execution memory.

```
foreach (var role in roles) { ... }
```

- **Line Purpose:** Iterates through each role name in the array to check for existence and create missing roles.
- **Keyword Breakdown:** `foreach` (loop construct that enumerates elements of a collection).
- **Data Flow:** Evaluates each system security role string sequentially.

```
var roleExists = await roleManager.RoleExistsAsync(role);
```

- **Line Purpose:** Queries the database to check if the current role name already exists in the `Roles` table.
- **Keyword Breakdown:** `await` (suspends method execution until the asynchronous task completes), `RoleExistsAsync` (checks if a role name exists in the database).
- **Cross-File References:** Executes a lookup via `ApplicationDbContext` against the `Roles` table.
- **Data Flow:** Sends an asynchronous `SELECT` query to PostgreSQL and returns a boolean value (`true`/`false`).

```
if (!roleExists) { await roleManager.CreateAsync(new IdentityRole<Guid>(role)); }
```

- **Line Purpose:** Conditional statement: if the role is missing, it initializes a new role object and saves it to the database.
- **Keyword Breakdown:** `if` (conditional statement checking truthiness), `CreateAsync` (persists a new identity role into the tracking store), `new IdentityRole<Guid>(role)` (instantiates a new role object assigned with the target name string).
- **Cross-File References:** Saves records directly to the `Roles` database table configured in `ApplicationDbContext`.
- **Data Flow:** Sends an `INSERT` statement to PostgreSQL to add the missing system security role record.

File 3: `EmployeeRepository.cs`

`using System.Data;`

- **Line Purpose:** Imports core .NET database interfaces, such as `IDbConnection` and `CommandType`.
- **Cross-File References:** Standard system library dependency.
- **Data Flow:** None.

`using Dapper;`

- **Line Purpose:** Imports Dapper, a lightweight micro-ORM used to execute raw SQL queries and map results directly to C# classes.
- **Cross-File References:** Third-party NuGet package framework tool dependency.
- **Data Flow:** None.

`using HRMS.Application.Interfaces;`

- **Line Purpose:** Imports repository application contract definitions to ensure this implementation meets business interface specifications.
- **Cross-File References:** Links directly to `IEmployeeRepository.cs` inside the **Application** project layer's `Interfaces/` folder.
- **Data Flow:** None.

`using HRMS.Domain.Entities;`

- **Line Purpose:** Imports the core entity definitions used to instantiate, return, or map employee data data models.
- **Cross-File References:** Links to `Employee.cs`, `ApplicationUser.cs`, and `Organization.cs` inside the **Domain** layer.
- **Data Flow:** None.

`using Microsoft.Extensions.Configuration;`

- **Line Purpose:** Imports the configuration abstractions used to read settings from `appsettings.json` (such as the database connection string).
- **Cross-File References:** Framework configuration dependency.
- **Data Flow:** None.

`using Npgsql;`

- **Line Purpose:** Imports the open-source .NET data provider for PostgreSQL to handle connections to the database.
- **Cross-File References:** NuGet database driver package reference.
- **Data Flow:** None.

```
namespace HRMS.Infrastructure.Repositories;
```

- **Line Purpose:** Groups data access repository implementation components into a dedicated logical namespace.
- **Cross-File References:** Scoped boundary used to organize data-access repositories.

```
public class EmployeeRepository : IEmployeeRepository
```

- **Line Purpose:** Defines the `EmployeeRepository` class, implementing the application layer contract for employee database operations.
- **Keyword Breakdown:** `implements` signifier `:`.
- **Cross-File References:** Implements the `IEmployeeRepository` contract from the **Application** layer. Registered as a dependency in `DependencyInjection.cs`.
- **Data Flow:** Coordinates all system data exchange interactions targeting employee table resources.

```
private readonly string _connectionString;
```

- **Line Purpose:** A private, immutable string property that stores the database connection string.
- **Keyword Breakdown:** `readonly` (can only be assigned during declaration or inside the constructor).
- **Data Flow:** Holds the security connection properties needed to access the PostgreSQL database instance.

```
public EmployeeRepository(IConfiguration configuration)
```

- **Line Purpose:** Constructor that injects the application configuration provider to read the required connection details.
- **Cross-File References:** Receives runtime configuration settings managed by `Program.cs`.
- **Data Flow:** Extracts the target database connection string from `appsettings.json` during initialization.

```
_connectionString = configuration.GetConnectionString("DefaultConnection") ?? throw new
InvalidOperationException("...");
```

- **Line Purpose:** Extracts the database connection string named `"DefaultConnection"`. If missing, it throws a runtime error to prevent application misconfigurations.
- **Keyword Breakdown:** `GetConnectionString` (helper technique returning specific items from the ConnectionStrings config section), `??` (null-coalescing operator executing the right side if the left side evaluates to null), `throw new` (raises a critical system runtime exception).
- **Data Flow:** Populates `_connectionString` with the database credentials or halts execution if they are missing.

```
private IDbConnection CreateConnection() => new NpgsqlConnection(_connectionString);
```

- **Line Purpose:** A helper method that instantiates and returns an active connection to the PostgreSQL database.
- **Keyword Breakdown:** `=>` (expression-bodied member syntax shortcut), `new NpgsqlConnection` (creates a connection instance utilizing the target database driver).
- **Data Flow:** Generates ready-to-open connection pipelines to execute SQL commands against PostgreSQL.

```
public async Task<Employee> CreateAsync(Employee employee)
```

- **Line Purpose:** Asynchronously calls a stored procedure to insert a new employee record and returns the created employee object.
- **Cross-File References:** Implements a contract from `IEmployeeRepository.cs`. Maps parameters from `Employee.cs`.
- **Data Flow:** Receives an employee entity domain object populated by the application layer.

```
const string procedureName = "sp_CreateEmployee";
```

- **Line Purpose:** Declares a local constant containing the name of the PostgreSQL stored procedure to execute.
- **Keyword Breakdown:** `const` (compile-time constant value).
- **Data Flow:** Holds the target procedure identifier string.

```
using var connection = CreateConnection();
```

- **Line Purpose:** Obtains a database connection instance that automatically closes and disposes when the method completes execution.
- **Keyword Breakdown:** `using var` (implicitly typed variable declaration with automatic disposal lifecycle management).
- **Data Flow:** Initializes a network connection context to the target database server.

```
await connection.ExecuteAsync(procedureName, new { ... }, commandType:
CommandType.StoredProcedure);
```

- **Line Purpose:** Uses Dapper to asynchronously execute the stored procedure, passing an anonymous object containing the procedure parameters.
- **Keyword Breakdown:** `ExecuteAsync` (Dapper extension method for write operations that returns the number of rows affected), `CommandType.StoredProcedure` (tells Dapper to execute the query as a stored procedure rather than a raw text query).
- **Data Flow:** Passes the employee property values to the `sp_CreateEmployee` procedure in PostgreSQL to insert a new row.

```
return employee;
```

- **Line Purpose:** Returns the created employee entity object back to the calling service layer.
- **Data Flow:** Sends the populated entity instance back up the execution pipeline.

```
public async Task<Employee?> GetByIdAsync(Guid id)
```

- **Line Purpose:** Queries the database for a specific employee by their unique ID, using a database function.
- **Keyword Breakdown:** `Employee?` (nullable reference type meaning it can return an Employee object or null if not found).
- **Data Flow:** Receives a lookup `Guid` parameter from the service layer.

```
const string sql = "SELECT * FROM sp_GetEmployeeById(@p_id)";
```

- **Line Purpose:** Defines the raw SQL query used to execute the table-returning PostgreSQL function.
- **Data Flow:** Passes the lookup key to the `sp_GetEmployeeById` function.

```
var row = await connection.QueryFirstOrDefaultAsync<EmployeeRow>(sql, new { p_id = id });
```

- **Line Purpose:** Uses Dapper to run the query and map the first row returned into an internal `EmployeeRow` data carrier object.
- **Keyword Breakdown:** `QueryFirstOrDefaultAsync<T>` (asynchronously executes a query and returns the first result, or a default null value if no rows match).
- **Data Flow:** Fetches database records into local memory and maps them to an `EmployeeRow` instance.

```
return row is null ? null : MapRowToEmployee(row);
```

- **Line Purpose:** Conditional ternary statement: if no row is found, it returns null. Otherwise, it maps the flat `EmployeeRow` object to a structured `Employee` domain entity.
- **Keyword Breakdown:** `is null` (pattern matching null check), `? :` (conditional ternary operator).
- **Data Flow:** Converts flat flat data rows into domain objects before passing them up to the service layer.

```
public async Task<List<Employee>> GetAllAsync(Guid? organizationId, Guid? createdBy)
```

- **Line Purpose:** Retrieves a list of employees, optionally filtered by organization ID or the user who created the record.
- **Keyword Breakdown:** `Guid?` (nullable unique identifier that can pass null to bypass filtering).
- **Data Flow:** Receives optional filter parameters to narrow the query results.

```
const string sql = "SELECT * FROM sp_GetAllEmployees(@p_organization_id, @p_created_by)";
```

- **Line Purpose:** Defines the raw SQL command to query the table-returning function `sp_GetAllEmployees`.
- **Data Flow:** Bundles the query parameters to pass them to the PostgreSQL function engine.

```
var rows = await connection.QueryAsync<EmployeeRow>(sql, new { p_organization_id =
organizationId, p_created_by = createdBy });
```

- **Line Purpose:** Uses Dapper to run the query and map the matching database rows into an in-memory collection of `EmployeeRow` objects.
- **Keyword Breakdown:** `QueryAsync<T>` (asynchronously executes a query and returns a collection of mapped objects).
- **Data Flow:** Streams matching database rows into an in-memory collection.

```
return rows.Select(MapRowToEmployee).ToList();
```

- **Line Purpose:** Uses LINQ to map each flat `EmployeeRow` object to a structured `Employee` entity and returns them as a standard list.
- **Keyword Breakdown:** `Select` (LINQ projection operator that maps each element in a collection), `ToList` (converts an enumerable sequence into a concrete `List<T>`).
- **Data Flow:** Projects flat database rows into domain object structures and returns the list.

```
public async Task<Employee> UpdateAsync(Employee employee)
```

- **Line Purpose:** Asynchronously invokes a stored procedure to update an existing employee record in the database.
- **Data Flow:** Receives an updated employee entity instance from the service layer.

```
const string procedureName = "sp_UpdateEmployee";
```

- **Line Purpose:** Defines a constant containing the name of the PostgreSQL stored procedure responsible for updates.

```
await connection.ExecuteAsync(procedureName, new { ... }, commandType:
CommandType.StoredProcedure);
```

- **Line Purpose:** Uses Dapper to execute the update stored procedure, passing an anonymous object containing the updated employee data.
- **Data Flow:** Updates the matching row values in the PostgreSQL database using the provided parameter properties.

```
return employee;
```

- **Line Purpose:** Returns the updated employee entity object back to the caller.

```
public async Task DeleteAsync(Employee employee)
```

- **Line Purpose:** Asynchronously executes a stored procedure to delete a specific employee record from the database.
- **Data Flow:** Receives the employee entity object targeted for deletion.

```
const string procedureName = "sp_DeleteEmployee";
```

- **Line Purpose:** Defines a constant containing the name of the PostgreSQL deletion stored procedure.

```
await connection.ExecuteAsync(procedureName, new { p_id = employee.Id }, commandType:
CommandType.StoredProcedure);
```

- **Line Purpose:** Uses Dapper to execute the deletion stored procedure, passing the employee's unique ID.
- **Data Flow:** Permanently deletes the matching employee row from the database using the provided `Id`.

```
private sealed class EmployeeRow
```

- **Line Purpose:** A private data structure used internally to map flat database rows before they are converted into structured domain models.
- **Keyword Breakdown:** `sealed` (prevents other classes from inheriting from this class), `class` (internal reference container type).
- **Data Flow:** Temporary data carrier that matches the columns returned by the PostgreSQL functions.

```
public Guid Id { get; init; } ... public string? OrganizationName { get; init; }
```

- **Line Purpose:** Declares properties on the internal `EmployeeRow` class that match the columns returned by the database.
- **Keyword Breakdown:** `init` (property accessor making it immutable after initialization).
- **Data Flow:** Temporarily holds raw column data, including joined fields like `CreatedByName` and `OrganizationName`.

```
private static Employee MapRowToEmployee(EmployeeRow row) => new() { ... };
```

- **Line Purpose:** A mapping method that transforms a flat `EmployeeRow` object into a structured `Employee` domain model, handling nested objects.
- **Keyword Breakdown:** `static` (method belongs to the class itself rather than a specific instance, preventing unnecessary memory allocations).
- **Data Flow:** Takes the flat database row data and constructs a structured `Employee` domain entity, populating its nested relational properties (`CreatedByUser` and `Organization`).

File 4: `HRRepository.cs`

```
using System.Data; using Dapper; using HRMS.Application.Interfaces; using
HRMS.Domain.Entities; using Microsoft.Extensions.Configuration; using Npgsql;
```

- **Line Purpose:** Imports the core libraries and custom project namespaces required for data access, mapping, and database connections.
- **Cross-File References:** Connects this file to standard system libraries, Dapper, PostgreSQL drivers, and the application interfaces contract layer (`IHRRepository.cs`).

```
namespace HRMS.Infrastructure.Repositories;
```

- **Line Purpose:** Declares the namespace boundary for the data access repository classes.

```
public class HRRepository : IHRRepository
```

- **Line Purpose:** Defines the `HRRepository` class, implementing the application layer contract for HR user data operations.
- **Cross-File References:** Implements the `IHRRepository` contract from the **Application** layer. Registered as a dependency in `DependencyInjection.cs`.

```
private readonly string _connectionString;
```

- **Line Purpose:** Stores the immutable database connection string.

```
public HRRepository(IConfiguration configuration)
```

- **Line Purpose:** Constructor that injects the application configuration provider to read settings from `appsettings.json`.

```
_connectionString = configuration.GetConnectionString("DefaultConnection") ?? throw new InvalidOperationException("...");
```

- **Line Purpose:** Extracts the connection string named `"DefaultConnection"`, throwing a runtime error if it is missing.

```
private IDbConnection CreateConnection() => new NpgsqlConnection(_connectionString);
```

- **Line Purpose:** A helper method that instantiates and returns an active connection to the PostgreSQL database.

```
public async Task<List<ApplicationUser>> GetHRsByOrganizationAsync(Guid organizationId)
```

- **Line Purpose:** Retrieves a list of users with the HR role belonging to a specific organization.
- **Data Flow:** Receives an `organizationId` parameter to filter the query results.

```
const string sql = "SELECT * FROM sp_GetHRsByOrganization(@p_organization_id)";
```

- **Line Purpose:** Defines the raw SQL query used to execute the table-returning PostgreSQL function `sp_GetHRsByOrganization`.

```
using var connection = CreateConnection();
```

- **Line Purpose:** Obtains a database connection instance that automatically closes and disposes when the method completes execution.

```
var rows = await connection.QueryAsync<HRUserRow>(sql, new { p_organization_id = organizationId });
```

- **Line Purpose:** Uses Dapper to run the query and map the matching database rows into an in-memory collection of internal `HRUserRow` objects.
- **Data Flow:** Sends the organization ID to the database function and streams the matching rows into local memory.

```
return rows.Select(MapRowToApplicationUser).ToList();
```

- **Line Purpose:** Maps the flat database rows into structured `ApplicationUser` domain entities and returns them as a standard list.
- **Data Flow:** Converts the flat database data into domain objects before passing them up to the calling service.

```
public async Task<ApplicationUser?> GetHRByIdAsync(Guid hrId)
```

- **Line Purpose:** Retrieves a specific HR user from the database by their unique ID.
- **Data Flow:** Receives an `hrId` parameter to locate the specific user.

```
const string sql = "SELECT * FROM sp_GetHRById(@p_hr_id)";
```

- **Line Purpose:** Defines the raw SQL query used to execute the table-returning PostgreSQL function `sp_GetHRById`.

```
var row = await connection.QueryFirstOrDefaultAsync<HRUserRow>(sql, new { p_hr_id = hrId });
```

- **Line Purpose:** Uses Dapper to execute the query and map the first result into an internal `HRUserRow` object, or returns null if no match is found.
- **Data Flow:** Queries the database for the matching user record and returns the flat data row.

```
return row is null ? null : MapRowToApplicationUser(row);
```

- **Line Purpose:** Conditional statement: if no user is found, it returns null. Otherwise, it maps the flat `HRUserRow` object to a structured `ApplicationUser` domain model.
- **Data Flow:** Converts the flat database row into an `ApplicationUser` entity instance and returns it.

```
private sealed class HRUserRow
```

- **Line Purpose:** A private data structure used internally to map flat database rows returned by the HR-related PostgreSQL functions.
- **Keyword Breakdown:** `sealed` (prevents inheritance for optimization).
- **Data Flow:** Temporarily holds raw column data returned by the database queries.

```
public Guid Id { get; init; } ... public DateTime UpdatedAt { get; init; }
```

- **Line Purpose:** Declares properties on the internal `HRUserRow` class that match the columns returned by the database functions.
- **Data Flow:** Temporarily stores raw field data like `FullName`, `Email`, and audit timestamps in local memory.

```
private static ApplicationUser MapRowToApplicationUser(HRUserRow row) => new() { ... };
```

- **Line Purpose:** A mapping method that transforms a flat `HRUserRow` object into a structured `ApplicationUser` domain entity.
- **Data Flow:** Copies data from the flat row object into a new instance of `ApplicationUser`, setting properties like `UserName` to match the email.

File 5: `OrganizationRepository.cs`

```
using System.Data; using Dapper; using HRMS.Application.Interfaces; using
HRMS.Domain.Entities; using Microsoft.Extensions.Configuration; using Npgsql;
```

- **Line Purpose:** Imports the core libraries and custom project namespaces required for data access, mapping, and database connections.
- **Cross-File References:** Connects this file to standard system libraries, Dapper, PostgreSQL drivers, and the application interfaces contract layer (`IOrganizationRepository.cs`).

```
namespace HRMS.Infrastructure.Repositories;
```

- **Line Purpose:** Declares the namespace boundary for the data access repository classes.

```
public class OrganizationRepository : IOrganizationRepository
```

- **Line Purpose:** Defines the `OrganizationRepository` class, implementing the application layer contract for organization database operations.
- **Cross-File References:** Implements the `IOrganizationRepository` contract from the **Application** layer. Registered as a dependency in `DependencyInjection.cs`.

```
private readonly string _connectionString;
```

- **Line Purpose:** Stores the immutable database connection string.

```
public OrganizationRepository(IConfiguration configuration)
```

- **Line Purpose:** Constructor that injects the application configuration provider to read settings from `appsettings.json`.

```
_connectionString = configuration.GetConnectionString("DefaultConnection") ?? throw new InvalidOperationException("...");
```

- **Line Purpose:** Extracts the connection string named `"DefaultConnection"`, throwing a runtime error if it is missing.

```
private IDbConnection CreateConnection() => new NpgsqlConnection(_connectionString);
```

- **Line Purpose:** A helper method that instantiates and returns an active connection to the PostgreSQL database.

```
public async Task<Organization> CreateAsync(Organization organization)
```

- **Line Purpose:** Asynchronously invokes a stored procedure to insert a new organization record into the database.
- **Data Flow:** Receives a new organization entity object from the service layer.

```
const string procedureName = "sp_CreateOrganization";
```

- **Line Purpose:** Defines a constant containing the name of the PostgreSQL stored procedure responsible for creating organizations.

```
using var connection = CreateConnection();
```

- **Line Purpose:** Obtains a database connection instance that automatically closes and disposes when the method completes execution.

```
await connection.ExecuteAsync(procedureName, new { ... }, commandType: CommandType.StoredProcedure);
```

- **Line Purpose:** Uses Dapper to execute the stored procedure, passing an anonymous object containing the organization data.
- **Data Flow:** Passes the organization properties to the `sp_CreateOrganization` procedure in PostgreSQL to insert a new row.

```
return organization;
```

- **Line Purpose:** Returns the created organization entity object back to the caller.

```
public async Task<List<Organization>> GetAllAsync()
```

- **Line Purpose:** Retrieves a list of all organizations from the database using a database function.
- **Data Flow:** Streams all organization records from the database into local memory.

```
const string sql = "SELECT * FROM sp_GetAllOrganizations()";
```

- **Line Purpose:** Defines the raw SQL query used to execute the table-returning PostgreSQL function `sp_GetAllOrganizations`.

```
var rows = await connection.QueryAsync<OrganizationRow>(sql);
```

- **Line Purpose:** Uses Dapper to run the query and map the returned database rows into an in-memory collection of internal `OrganizationRow` objects.
- **Data Flow:** Fetches database records into local memory and maps them to an array of `OrganizationRow` objects.

```
return rows.Select(MapRowToOrganization).ToList();
```

- **Line Purpose:** Maps the flat database rows into structured `Organization` domain entities and returns them as a standard list.
- **Data Flow:** Converts flat data rows into domain objects before passing them up to the calling service.

```
public async Task<Organization?> GetByIdAsync(Guid id)
```

- **Line Purpose:** Retrieves a specific organization from the database by its unique ID.
- **Data Flow:** Receives an `id` parameter to locate the specific organization record.

```
const string sql = "SELECT * FROM sp_GetOrganizationById(@p_id)";
```

- **Line Purpose:** Defines the raw SQL query used to execute the table-returning PostgreSQL function `sp_GetOrganizationById`.

```
var row = await connection.QueryFirstOrDefaultAsync<OrganizationRow>(sql, new { p_id = id });
```

- **Line Purpose:** Uses Dapper to execute the query and map the first result into an internal `OrganizationRow` object, or returns null if no match is found.
- **Data Flow:** Queries the database for the matching organization record and returns the flat data row.

```
return row is null ? null : MapRowToOrganization(row);
```

- **Line Purpose:** Conditional statement: if no organization is found, it returns null. Otherwise, it maps the flat `OrganizationRow` object to a structured `Organization` domain model.
- **Data Flow:** Converts the flat database row into an `Organization` entity instance and returns it.

```
private sealed class OrganizationRow
```

- **Line Purpose:** A private data structure used internally to map flat database rows returned by the organization-related PostgreSQL functions.
- **Data Flow:** Temporarily holds raw column data returned by the database queries.

```
public Guid Id { get; init; } ... public DateTime CreatedAt { get; init; }
```

- **Line Purpose:** Declares properties on the internal `OrganizationRow` class that match the columns returned by the database functions.
- **Data Flow:** Temporarily stores raw field data like `Name`, `Address`, and creation timestamps in local memory.

```
private static Organization MapRowToOrganization(OrganizationRow row) => new() { ... };
```

- **Line Purpose:** A mapping method that transforms a flat `OrganizationRow` object into a structured `Organization` domain entity.
- **Data Flow:** Copies data from the flat row object into a new instance of `Organization` and returns it.

File 6: `RefreshTokenRepository.cs`

```
using System.Data; using Dapper; using HRMS.Application.Interfaces; using
HRMS.Domain.Entities; using Microsoft.Extensions.Configuration; using Npgsql;
```

- **Line Purpose:** Imports the core libraries and custom project namespaces required for data access, mapping, and database connections.
- **Cross-File References:** Connects this file to standard system libraries, Dapper, PostgreSQL drivers, and the application interfaces contract layer (`IRefreshTokenRepository.cs`).

```
namespace HRMS.Infrastructure.Repositories;
```

- **Line Purpose:** Declares the namespace boundary for the data access repository classes.

```
public class RefreshTokenRepository : IRefreshTokenRepository
```

- **Line Purpose:** Defines the `RefreshTokenRepository` class, implementing the application layer contract for refresh token database operations.
- **Cross-File References:** Implements the `IRefreshTokenRepository` contract from the **Application** layer. Registered as a dependency in `DependencyInjection.cs`.

private readonly string _connectionString;

- **Line Purpose:** Stores the immutable database connection string.

```
public RefreshTokenRepository(IConfiguration configuration)
```

- **Line Purpose:** Constructor that injects the application configuration provider to read settings from `appsettings.json`.

```
_connectionString = configuration.GetConnectionString("DefaultConnection") ?? throw new
InvalidOperationException("...");
```

- **Line Purpose:** Extracts the connection string named `"DefaultConnection"`, throwing a runtime error if it is missing.

```
private IDbConnection CreateConnection() => new NpgsqlConnection(_connectionString);
```

- **Line Purpose:** A helper method that instantiates and returns an active connection to the PostgreSQL database.

```
public async Task AddAsync(RefreshToken token)
```

- **Line Purpose:** Asynchronously invokes a stored procedure to insert a new refresh token record into the database.
- **Data Flow:** Receives a new refresh token entity object from the service layer.

```
const string procedureName = "sp_AddRefreshToken";
```

- **Line Purpose:** Defines a constant containing the name of the PostgreSQL stored procedure responsible for adding refresh tokens.

```
using var connection = CreateConnection();
```

- **Line Purpose:** Obtains a database connection instance that automatically closes and disposes when the method completes execution.

```
await connection.ExecuteAsync(procedureName, new { ... }, commandType:
CommandType.StoredProcedure);
```

- **Line Purpose:** Uses Dapper to execute the stored procedure, passing an anonymous object containing the token data.
- **Data Flow:** Passes the token properties to the `sp_AddRefreshToken` procedure in PostgreSQL to insert a new row.

```
public async Task<RefreshToken?> GetByTokenAsync(string token)
```

- **Line Purpose:** Retrieves a specific refresh token record from the database by its token string value.
- **Data Flow:** Receives a token string parameter to locate the specific record.

```
const string sql = "SELECT * FROM sp_GetRefreshTokenByToken(@p_token)";
```

- **Line Purpose:** Defines the raw SQL query used to execute the table-returning PostgreSQL function `sp_GetRefreshTokenByToken`.

```
var row = await connection.QueryFirstOrDefaultAsync<RefreshTokenRow>(sql, new { p_token = token });
```

- **Line Purpose:** Uses Dapper to execute the query and map the first result into an internal `RefreshTokenRow` object, or returns null if no match is found.
- **Data Flow:** Queries the database for the matching token record and returns the flat data row.

```
return row is null ? null : MapRowToRefreshToken(row);
```

- **Line Purpose:** Conditional statement: if no token is found, it returns null. Otherwise, it maps the flat `RefreshTokenRow` object to a structured `RefreshToken` domain model.
- **Data Flow:** Converts the flat database row into a structured `RefreshToken` entity (including its nested `User` property) and returns it.

```
public async Task UpdateAsync(RefreshToken token)
```

- **Line Purpose:** Asynchronously invokes a stored procedure to update an existing refresh token record (typically used to mark it as revoked).
- **Data Flow:** Receives the updated refresh token entity object from the service layer.

```
const string procedureName = "sp_UpdateRefreshToken";
```

- **Line Purpose:** Defines a constant containing the name of the PostgreSQL stored procedure responsible for updating tokens.

```
await connection.ExecuteAsync(procedureName, new { p_id = token.Id, p_revoked = token.Revoked }, commandType: CommandType.StoredProcedure);
```

- **Line Purpose:** Uses Dapper to execute the update stored procedure, passing the token's ID and its revocation status.
- **Data Flow:** Updates the matching token row in the database, setting its revocation timestamp.

```
public async Task RevokeAllUserTokensAsync(Guid userId)
```

- **Line Purpose:** Asynchronously invokes a stored procedure to revoke all active refresh tokens belonging to a specific user.
- **Data Flow:** Receives a `userId` parameter to target all tokens belonging to that user.

```
const string procedureName = "sp_RevokeAllUserTokens";
```

- **Line Purpose:** Defines a constant containing the name of the PostgreSQL stored procedure responsible for revoking all tokens for a user.

```
await connection.ExecuteAsync(procedureName, new { p_user_id = userId, p_revoked_at = DateTime.UtcNow }, commandType: CommandType.StoredProcedure);
```

- **Line Purpose:** Uses Dapper to execute the revocation stored procedure, passing the user ID and the current UTC timestamp.
- **Keyword Breakdown:** `DateTime.UtcNow` (gets a DateTime object that is set to the current date and time on this computer, expressed as the Coordinated Universal Time).
- **Data Flow:** Updates all matching token rows for the specified user in the database, setting their revocation timestamps to the current time.

```
private sealed class RefreshTokenRow
```

- **Line Purpose:** A private data structure used internally to map flat database rows returned by the token-related PostgreSQL functions.
- **Data Flow:** Temporarily holds raw column data, including joined user fields like `UserFullName` and `UserEmail`.

```
public Guid Id { get; init; } ... public Guid? UserOrgId { get; init; }
```

- **Line Purpose:** Declares properties on the internal `RefreshTokenRow` class that match the columns returned by the database functions.
- **Data Flow:** Temporarily stores raw field data and joined user properties in local memory.

```
private static RefreshToken MapRowToRefreshToken(RefreshTokenRow row) => new() { ... };
```

- **Line Purpose:** A mapping method that transforms a flat `RefreshTokenRow` object into a structured `RefreshToken` domain entity, handling the nested user object.
- **Data Flow:** Takes the flat database row data and constructs a structured `RefreshToken` domain entity, populating its nested `User` property with an `ApplicationUser` instance containing the joined user details.

File 7: `DependencyInjection.cs`

```
using HRMS.Application.Interfaces; using HRMS.Domain.Entities; using
HRMS.Infrastructure.Data; using HRMS.Infrastructure.Repositories; using
Microsoft.AspNetCore.Identity; using Microsoft.EntityFrameworkCore; using
Microsoft.Extensions.Configuration; using Microsoft.Extensions.DependencyInjection;
```

- **Line Purpose:** Imports all the necessary internal layers (Interfaces, Domain, Data Context, Repositories) and external framework dependencies required to register application services.
- **Cross-File References:** Links all individual layers of your architecture together within this setup orchestration file.

```
namespace HRMS.Infrastructure;
```

- **Line Purpose:** Declares the namespace boundary for the Infrastructure project's configuration classes.

```
public static class DependencyInjection
```

- **Line Purpose:** Declares a static configuration class that extends `IServiceCollection` to encapsulate infrastructure service registrations.
- **Keyword Breakdown:** `static` (indicates a class that can contain only static extension configuration methods).
- **Cross-File References:** This class's extension method is called inside the root initialization file `Program.cs`.

```
public static IServiceCollection AddInfrastructure(this IServiceCollection services,
IConfiguration configuration)
```

- **Line Purpose:** Defines an extension method that registers all infrastructure services (Database context, Identity framework, and Repositories) into the application's service container.
- **Keyword Breakdown:** `IServiceCollection` (the container interface for registering application dependencies), `this` (modifies the first parameter, making this an extension method that can be called directly on an `IServiceCollection` instance).
- **Cross-File References:** Returns the container instance back to `Program.cs` to allow chaining further service configurations.
- **Data Flow:** Receives the central service collection and configuration settings, registers dependencies into it, and returns it.

```
var connectionString = configuration.GetConnectionString("DefaultConnection");
```

- **Line Purpose:** Extracts the database connection string from the configuration manager.

- **Data Flow:** Reads the connection settings from `appsettings.json` and stores the string value in a local variable.

```
services.AddDbContext<ApplicationDbContext>(options => options.UseNpgsql(connectionString, b => b.MigrationsAssembly(typeof(ApplicationDbContext).Assembly.FullName)));
```

- **Line Purpose:** Registers your custom `ApplicationDbContext` within the dependency injection container, configuring it to connect to a PostgreSQL database and specifying where database migrations should be generated.
- **Keyword Breakdown:** `AddDbContext<T>` (registers the DbContext type as a Scoped service), `UseNpgsql` (configures EF Core to use the PostgreSQL database provider), `MigrationsAssembly` (specifies the project assembly where database migrations are stored).
- **Cross-File References:** Registers `ApplicationDbContext.cs` from this folder into the system container.
- **Data Flow:** Provisions the application with transactional context factories used for generating or looking up structural database migrations.

```
services.AddIdentity<ApplicationUser, IdentityRole<Guid>>(options => { ... })
```

- **Line Purpose:** Registers and configures the ASP.NET Core Identity framework system using your custom user and role types.
- **Keyword Breakdown:** `AddIdentity<TUser, TRole>` (adds core identity system services to the application container).
- **Cross-File References:** Links configuration rules directly onto `ApplicationUser.cs` from **Domain** and sets up underlying dependencies for `IdentitySeed.cs`.
- **Data Flow:** Loads custom authentication, authorization, and validation rules into the global application engine.

```
options.Password.RequireDigit = false; options.Password.RequiredLength = 6; ... options.User.RequireUniqueEmail = true;
```

- **Line Purpose:** Configures security rules for user accounts, loosening standard password requirements (no required numbers, capital letters, or special characters; minimum length of 6) and forcing emails to be unique across all accounts.
- **Data Flow:** Configures internal validation engines to enforce these exact security constraints when creating or updating users.

```
.AddEntityFrameworkStores<ApplicationDbContext>().AddDefaultTokenProviders();
```

- **Line Purpose:** Tells the Identity framework to use Entity Framework Core and your `ApplicationDbContext` to save its data, and adds default token generators for operations like password resets.
- **Keyword Breakdown:** `AddEntityFrameworkStores` (configures Identity to use an EF Core data store), `AddDefaultTokenProviders` (adds providers that generate security tokens for operations like password changes or multi-factor authentication).
- **Cross-File References:** Binds the Identity system directly to the schema definitions inside `ApplicationDbContext.cs`.
- **Data Flow:** Connects the user management system to your database tables and provisions the system with security token generation tools.

```
services.AddScoped<IOrganizationRepository, OrganizationRepository>();
```

- **Line Purpose:** Registers the organization repository under a Scoped lifetime, meaning a new instance is created once per HTTP request.
- **Keyword Breakdown:** `AddScoped<TInterface, TImplementation>` (registers a dependency with a scoped lifetime linked to the lifecycle of an incoming web request).
- **Cross-File References:** Maps the interface `IOrganizationRepository.cs` (from the **Application** layer) to its implementation `OrganizationRepository.cs` (in this folder).

- **Data Flow:** Allows the API controllers or application services to request `IOrganizationRepository` and automatically receive a running instance of `OrganizationRepository`.

```
services.AddScoped<IEmployeeRepository, EmployeeRepository>();
```

- **Line Purpose:** Registers the employee repository under a Scoped lifetime.
- **Cross-File References:** Maps the interface `IEmployeeRepository.cs` (from the **Application** layer) to its implementation `EmployeeRepository.cs` (in this folder).

```
services.AddScoped<IHRRepository, HRRepository>();
```

- **Line Purpose:** Registers the HR repository under a Scoped lifetime.
- **Cross-File References:** Maps the interface `IHRRepository.cs` (from the **Application** layer) to its implementation `HRRepository.cs` (in this folder).

```
services.AddScoped<IRefreshTokenRepository, RefreshTokenRepository>();
```

- **Line Purpose:** Registers the refresh token repository under a Scoped lifetime.
- **Cross-File References:** Maps the interface `IRefreshTokenRepository.cs` (from the **Application** layer) to its implementation `RefreshTokenRepository.cs` (in this folder).

```
return services;
```

- **Line Purpose:** Returns the modified service container instance back to the caller.
- **Data Flow:** Passes the updated container back to `Program.cs` to complete application startup configuration.

